Sealed Trade Protocol: A Peer-to-Peer Bilateral Trade System with Information-Sealed Negotiation

Shota Nagafuchi

sealed-trade@proton.me

ABSTRACT

We propose a protocol for bilateral trade that prevents the double-use of private information during negotiation. In conventional private markets, the act of negotiating reveals private valuations which counterparties exploit to extract surplus. A buyer's willingness to pay, once signaled through an offer, can never be unsignaled. A seller's urgency, once revealed through a concession, permanently weakens their position. Sealed Trade addresses this by confining negotiation to AI agents operating within hardware-isolated enclaves. Each agent is bound by cryptographically signed parameters from its principal, and provably destroys all negotiation state upon completion. Only the final outcome — agreement or no deal — crosses the enclave boundary. Settlement occurs on-chain via bonded contracts with a volume-based mining mechanism that bootstraps early adoption.

1. Introduction

Private bilateral markets — IP licensing, M&A, real estate — require negotiation to reach agreement. But negotiation itself is a destructive act: every offer, counteroffer, and hesitation reveals private information that the counterparty can exploit. The information required to reach a deal is the same information used to worsen its terms.

We call this the **double-use of private information**. When two parties negotiate, each party's private information — their reservation price, urgency, alternatives — is necessarily communicated during the process. Once communicated, this information can be used by the counterparty to extract surplus beyond what a fair exchange would produce.

Consider a patent licensing negotiation. The licensee knows the maximum they would pay; the licensor knows the minimum they would accept. In an ideal negotiation, they would agree on a price within this zone of possible agreement (ZOPA) without either party learning the other's boundary. In practice, every offer reveals information. A first offer of $100K signals willingness to pay at least $100K.

A counteroffer of $500K signals willingness to accept at most $500K. Through iterated rounds, each party's private valuation is progressively disclosed and exploited.

This problem has been studied in mechanism design since Myerson and Satterthwaite [1], who proved that no incentive-compatible mechanism can achieve ex-post efficiency in bilateral trade with private valuations. Their impossibility result assumes that agents act strategically — which they must, because revealing truthful valuations is dominated by misrepresenting them.

We circumvent this impossibility not by designing a new mechanism, but by changing the information architecture. If negotiation occurs between AI agents in a hardware-sealed environment, and all intermediate state is provably destroyed afterward, the information double-use problem disappears. Each party's private valuation is used exactly once — by their own agent, to negotiate — and is never available to the counterparty.

2. The Information Double-Use Problem

2.1 Definition

In bilateral trade, private information serves two conflicting purposes:

1. **Reaching agreement**: Both parties must communicate preferences to find mutually acceptable terms.
2. **Maximizing individual surplus**: Each party wants to conceal preferences to prevent the counterparty from extracting value.

The fundamental tension is that the information required for (1) is the same information exploited in (2). We call this **information double-use**: private information that is used once to facilitate a deal is reused to worsen its terms.

2.2 Three Layers

Layer 1: Discovery leakage. Expressing interest in an asset reveals demand. A buyer who contacts a patent holder has revealed that the patent has strategic value to them. The seller can adjust pricing upward before any negotiation begins.

Layer 2: Negotiation extraction. Each offer and counteroffer is a signal. Anchoring effects, response timing, concession patterns, and even the choice to continue negotiating all leak information about private valuations. Experienced negotiators exploit these signals systematically.

Layer 3: Post-settlement persistence. After a deal closes, the intermediary retains complete knowledge of both parties' reservation prices and negotiation behavior. This information can be used in future deals, shared with other clients, or leveraged in related transactions.

2.3 Inadequacy of Existing Approaches

Trusted intermediaries promise confidentiality but have no technical enforcement mechanism. Their business model depends on information asymmetry — they profit from knowing what both sides will accept.

Multi-party computation [2] can protect specific computations but cannot support the open-ended, natural-language negotiation required in complex bilateral deals. MPC protocols require predefined computation circuits; free-form LLM-based negotiation cannot be expressed as a circuit.

Fully homomorphic encryption [3] imposes computational overhead several orders of magnitude beyond what is feasible for LLM inference, making it impractical for the scale of computation involved in agent-based negotiation.

Zero-knowledge proofs prove computation correctness but cannot seal arbitrary content. A ZKP can prove that an agent followed its parameters, but it cannot prevent the counterparty from observing the negotiation itself.

Dark pools [4][5] solve information leakage for fungible token swaps — matching buyers and sellers without revealing order flow. But bilateral trade involves non-fungible, complex assets that require multi-dimensional negotiation, not simple price matching.

3. Protocol Design

3.1 Trade Lifecycle

A trade progresses through six states:

```
Listed → Matched → Negotiating → Agreed → Settled
                                        ↗
                    (any state) → Cancelled
```

Listed. A seller publishes a hashed asset description and a maximum deal value. They post a Discovery bond. No information about the asset's nature is revealed — only that something is available.

Matched. A buyer expresses interest and posts a matching bond. Neither party can see the other's parameters.

Negotiating. Both parties escalate their bonds. Each party signs a parameter set defining acceptable terms for their AI agent:

```
Parameters = {
    price_range: [min, max],
    required_terms: [...],
    deal_breakers: [...],
    strategy: "..."
}

signature = sign(hash(Parameters), private_key)
```

The signed parameters are loaded into a hardware-isolated enclave. The agents negotiate via a structured message protocol. Neither agent can communicate outside the enclave. Neither principal can observe the negotiation.

Agreed. If the agents reach agreement, both parties escalate to the highest bond tier. The agreement hash and enclave attestation are recorded on the settlement layer.

Settled. The deal value transfers from buyer to seller, minus the protocol fee. Bonds are returned. Mining rewards are distributed.

Cancelled. At any point before agreement, either party can cancel. Before negotiation, bonds are returned without penalty. During or after negotiation, the cancelling party's bond is slashed: half to the counterparty as compensation, half to the protocol's insurance pool.

3.2 Information Sealing via Hardware Enclaves

Agents run inside Trusted Execution Environments (Intel TDX, AMD SEV-SNP). The enclave provides three properties:

1. **Memory isolation.** The enclave's memory is encrypted in hardware. No process, operating system, hypervisor, or physical access can read it.
2. **Remote attestation.** The enclave generates a cryptographic proof that specific code is running in an authentic environment. Counterparties verify that the agent code has not been tampered with.
3. **Attestation of destruction.** Upon negotiation completion, the enclave attests that all memory has been zeroed. This is a kernel-level operation — forensic recovery is infeasible.

3.3 Trust Assumption

The enclave requires trusting the hardware vendor to correctly implement isolation. We argue this assumption is comparable to standard cryptographic assumptions:

- Using ECDSA requires trusting that the discrete logarithm problem is hard.
- Using a hash function requires trusting collision resistance.
- Using a hardware enclave requires trusting that the vendor's isolation implementation is correct.

All three are empirically validated assumptions that could theoretically fail. The protocol bounds the economic consequence of failure through an insurance pool (see Section 5).

3.4 Agent Design

Each agent is a language model running inside the enclave. The agent receives signed parameters from its principal, negotiates with the counterparty's agent using structured messages, evaluates offers against its parameter boundaries, and either reaches agreement or declares no-deal.

The principal's signed parameters act as an **irrevocable mandate**. The agent cannot exceed them. If the principal sets a maximum price, the agent cannot agree to a higher one, regardless of the negotiation dynamics. This eliminates the agent alignment problem — the agent is not optimizing for its own objective but strictly executing within signed bounds.

4. Economic Model

4.1 Token Supply

The protocol's native token has a fixed supply of 100,000,000 units, allocated as:

- **95%** (95,000,000) to mining — distributed to trade participants over time
- **5%** (5,000,000) to treasury — for governance and ecosystem development

All tokens are created at protocol deployment. There is no inflation and no mechanism to create additional supply.

4.2 Volume-Based Halving

Mining rewards decrease as cumulative trade volume grows, following a halving schedule indexed by volume rather than time.

Let V denote cumulative trade volume. Period n spans the volume interval $(V_{n-1}, V_n]$, where:

$$V_n = H \cdot 2^n$$

and H = \$10,000 is the halving constant. The token allocation for period n is:

$$A_n = A_0 / 2^n$$

where A_0 = 47,500,000. The volume within period n is:

$\Delta V_n = H$ for $n = 0$, and $H \cdot 2^{n-1}$ for $n \geq 1$.

The mining rate (tokens per dollar traded) in period n is:

$$r_n = A_n / \Delta V_n$$

Period	Threshold	Allocation	Rate
0	$10K	47,500,000	4,750
1	$20K	23,750,000	2,375
2	$40K	11,875,000	593.75
3	$80K	5,937,500	148.44

The geometric series converges: $\Sigma A_n = A_0 \cdot (1 - 2^{-22}) / (1 - 2^{-1}) \approx 95{,}000{,}000$. Total cumulative volume to exhaust mining: approximately $21 billion.

4.3 Contribution Scoring

When a trade settles at value d, both buyer and seller receive equal contribution scores:

$$s_{buyer} = s_{seller} = d / 2$$

Within each period n, a contributor's reward is proportional to their share of the period's total score:

$$R_i^{(n)} = A_n \cdot s_i^{(n)} / \Sigma_j s_j^{(n)}$$

This ensures that the full period allocation is distributed to participants.

4.4 Bonds

Bonds serve two purposes: preventing spam and providing economic recourse for counterparty harm. The bond amount at each stage is:

$$B(d, stage) = clamp(d \cdot r_{stage}, B_{min}, B_{max})$$

where clamp(x, a, b) = min(max(x, a), b).

Stage	Rate	Minimum	Maximum
Discovery	1%	$1	$1,000
Negotiation	3%	$5	$5,000
Execution	10%	$10	$50,000

Escalation is additive: progressing from one stage to the next requires posting only the difference, not the full amount. On dispute, the faulty party's bond is split equally between the counterparty and the insurance pool.

4.5 Fee

A 0.3% fee is collected at settlement. The fee is paid by the buyer and deducted from the seller's proceeds.

5. Security Analysis

5.1 Enclave Breach

The protocol's confidentiality depends on hardware enclave integrity. We model breach probability as:

$$P(\text{breach}) = P(\text{hardware vulnerability}) \times P(\text{exploit within trade window}) \times P(\text{targeted at specific trade})$$

For current enclave technology, we estimate:

$$P(\text{breach}) \approx 10^{-3} \times 10^{-2} \times 10^{-2} = 10^{-7} \text{ per trade}$$

The expected loss per trade is therefore:

$$E[\text{loss}] = 10^{-7} \times \text{deal value}$$

For a $100,000 trade, the expected loss is $0.01. The insurance pool (funded by 50% of slashed bonds) is sized to cover aggregate expected losses across all trades.

5.2 Economic Attacks

Bond griefing. An attacker could repeatedly enter trades and cancel during negotiation to trigger bond slashing. The cost of this attack is the attacker's own bond (3% of deal value at minimum), while the benefit is the counterparty's wasted time. The minimum bond ensures that even small-value griefing has non-trivial cost.

Volume inflation. A participant could report inflated deal values to mine additional tokens. The cost of this attack is the bond required at each stage (up to 10% of the stated deal value). For deal values above the maximum bond cap ($50,000), the attacker's bond is fixed while their mining reward scales — creating diminishing attack cost per mined token at very high stated values. This is bounded by the maximum bond caps.

Sybil mining. A participant could create multiple identities to accumulate disproportionate mining rewards. Since rewards are proportional to contribution score (which is proportional to deal value), and each deal requires real bond capital, the cost of Sybil mining equals the bond capital required — which is proportional to the deal value being mined. There is no amplification.

5.3 Settlement Integrity

The settlement layer enforces a strict state machine. Each transition requires:

- Bond escalation from the transacting parties
- Cryptographic signatures from both buyer and seller (for agreement)
- Enclave attestation (for negotiation initiation and completion)

No transition can bypass these requirements. The state machine is deterministic and fully specified — there are no administrative overrides or upgrade mechanisms for the core trade logic.

6. Limitations

Hardware vendor trust. The protocol's confidentiality guarantee depends on enclave vendor integrity. A fundamental hardware compromise — not just a software exploit — would violate the sealed negotiation property. The insurance pool mitigates but does not eliminate this risk.

Self-reported deal values. The protocol does not independently verify that stated deal values reflect actual economic reality. Bond requirements create a cost proportional to misreporting, but the maximum bond caps create a ceiling on this cost.

Single-vertical cold start. The protocol requires participants in each trade vertical. Network effects within a vertical do not transfer unless participants overlap across verticals.

Agent capability. The quality of negotiation depends on the language model's ability to negotiate effectively within the parameter space. Agent capability is bounded by current model limitations.

7. Conclusion

We have presented a protocol that eliminates the double-use of private information in bilateral trade. The mechanism confines negotiation to hardware-isolated AI agents that provably destroy all intermediate state, ensuring that private valuations are used exactly once — by their owner's agent — and are never available to the counterparty, the intermediary, or the protocol itself.

A reference implementation is available as open-source software.

References

[1] R. Myerson and M. Satterthwaite, "Efficient Mechanisms for Bilateral Trading," *Journal of Economic Theory*, vol. 29, pp. 265–281, 1983.

[2] R. Moraffah and B. Sankar, "Privacy-Preserving Multi-Round Negotiations," *IEEE Transactions on Information Forensics and Security*, 2021.

[3] C. Gentry, "Fully Homomorphic Encryption Using Ideal Lattices," *STOC*, 2009.

[4] Renegade, "On-Chain Dark Pool," 2023.

[5] Penumbra Labs, "A Private DEX and Shielded Pool," 2023.